

Week 2 — Python & GenAI Capstone Rubric

Multi-Agent RAG System for Enterprise Documentation

Overall Evaluation
[Exceeds Expectations / Meets Expectations / Needs Improvement]

Instructor's Qualitative Observations:

- [Provide overall summary feedback here. Note strengths and key areas for growth based on the detailed evaluation below. e.g., "The qualitative RAG agent was exceptionally well-implemented, with accurate source attribution. The next step is to build out a comprehensive test suite to ensure the multi-agent workflows are robust and reliable."]*

Evaluation Dimensions

This rubric evaluates the project across three key dimensions.

- To **Meet Expectations**, a participant must satisfy the baseline **Completion** criteria and achieve "Meets Expectations" on all core **Concept Mastery** and **Process/Quality** items.
- Exceeds Expectations** is awarded for demonstrating advanced skills in multi-agent integration and automated testing.

1. Completion (Baseline Requirement)

Did the participant deliver a functional, end-to-end system? This is the pass/fail baseline for the project.

Status	Core Requirement
[Pass / Fail]	Runnable System: The project runs from a clean checkout using Python 3.8+. Dependencies are pinned in requirements.txt or pyproject.toml and installed correctly.
[Pass / Fail]	End-to-End Query Handling: The system can be started via a command-line interface, accept a user query, and return a complete answer from at least two of the three required agents.

If either of the above criteria is marked as "Fail," the project is considered "Needs Improvement" overall.

2. Concept Mastery

Did the participant demonstrate a strong grasp of the core GenAI and multi-agent architecture concepts?

Evaluation	Key Concept	Instructor Notes
[Needs Improvement / Meets Expectations / Exceeds Expectations]	Agent Orchestration (Manager): The Manager agent correctly classifies incoming queries, routes them to the appropriate specialized agent, and coordinates the final response. Handles unsupported queries gracefully.	[e.g., The routing logic is effective for simple queries, but struggles with complex, multi-part questions.]
[Needs Improvement / Meets Expectations / Exceeds Expectations]	Qualitative RAG Agent: Integrates with a vector database and an LLM to retrieve relevant document chunks. Answers include accurate citations or URLs pointing to the source documentation.	[e.g., Source attribution is excellent. The next step would be to experiment with different chunking strategies to improve retrieval accuracy.]
[Needs Improvement / Meets Expectations / Exceeds Expectations]	Quantitative NL-to-SQL Agent: Connects to a SQL database, translates a natural language question into a valid SQL query, executes it, and returns a formatted numeric or tabular answer.	[e.g., The agent successfully handles single-table queries, but fails on queries requiring joins.]
[Needs Improvement / Meets Expectations / Exceeds Expectations]	Full Three-Agent Integration (Stretch): All three agents (Manager, Qualitative, Quantitative) operate together seamlessly. The system automatically handles all query types without errors. (Required for Exceeds	[e.g., The three-agent system is fully functional and robust, representing a production-ready architecture.]

	Expectations)	
--	---------------	--

3. Process & Quality

Did the participant demonstrate essential software engineering and MLOps practices?

Evaluation	Engineering Practice	Instructor Notes
[Needs Improvement / Meets Expectations / Exceeds Expectations]	Documentation (README): The README provides a clear explanation of the system architecture, the role of each agent, and step-by-step setup and usage instructions. Includes an architecture diagram.	[e.g., The setup instructions are clear, but the README would benefit from a diagram illustrating how the agents interact.]
[Needs Improvement / Meets Expectations / Exceeds Expectations]	Command-Line Interface: The CLI is user-friendly, providing clear prompts for input, displaying formatted output, and offering a clean way to exit the application.	[e.g., The CLI is functional but could be improved with better error handling for invalid user inputs.]
[Needs Improvement / Meets Expectations / Exceeds Expectations]	Automated Testing (Stretch): A comprehensive test suite (e.g., pytest) is implemented, covering agent-level unit tests, multi-agent integration tests, and CLI behavior. (Required for Exceeds Expectations)	[e.g., Excellent test coverage across all agents. The integration tests effectively validate the end-to-end workflows.]